

Runtime

class Runtime `dataclass`

Bases: `Generic[ContextT]`

Convenience class that bundles run-scoped context and other runtime utilities.

 **Added in version v0.6.0**

Example:

```
from typing import TypedDict
from langgraph.graph import StateGraph
from dataclasses import dataclass
from langgraph.runtime import Runtime
from langgraph.store.memory import InMemoryStore

@dataclass
class Context: ❶
    user_id: str

class State(TypedDict, total=False):
    response: str

store = InMemoryStore() ❷
store.put(("users",), "user_123", {"name": "Alice"})

def personalized_greeting(state: State, runtime: Runtime[Context]) -> State:
    '''Generate personalized greeting using runtime context and store.'''
    user_id = runtime.context.user_id ❸
    name = "unknown_user"
    if runtime.store:
        if memory := runtime.store.get(("users",), user_id):
            name = memory.value["name"]

    response = f"Hello {name}! Nice to see you again."
    return {"response": response}
```

```
graph = (
    StateGraph(state_schema=State, context_schema=Context)
    .add_node("personalized_greeting", personalized_greeting)
    .set_entry_point("personalized_greeting")
    .set_finish_point("personalized_greeting")
    .compile(store=store)
)

result = graph.invoke({}, context=Context(user_id="user_123"))
print(result)
# > {'response': 'Hello Alice! Nice to see you again.'}
```

- 1 Define a schema for the runtime context.
- 2 Create a store to persist memories and other information.
- 3 Use the runtime context to access the `user_id`.

attr `context` class-attribute instance-attribute

```
context: ContextT = field(default=None)
```

Static context for the graph run, like `user_id`, `db_conn`, etc.

Can also be thought of as 'run dependencies'.

attr `store` class-attribute instance-attribute

```
store: BaseStore | None = field(default=None)
```

Store for the graph run, enabling persistence and memory.

attr `stream_writer` class-attribute instance-attribute

```
stream_writer: StreamWriter = field(default=_no_op_stream_writer)
```

Function that writes to the custom stream.

attr `previous` class-attribute instance-attribute

```
previous: Any = field(default=None)
```

The previous return value for the given thread.

Only available with the functional API when a checkpointer is provided.

mod runtime

FUNCTION	DESCRIPTION
<code>get_runtime</code>	Get the runtime for the current graph run.

func get_runtime

```
get_runtime(context_schema: type[ContextT] | None = None) -> Runtime[ContextT]
```

Get the runtime for the current graph run.

PARAMETER	DESCRIPTION
<code>context_schema</code>	Optional schema used for type hinting the return type of the runtime. TYPE: <code>type[ContextT]</code> <code>None</code> DEFAULT: <code>None</code>

RETURNS	DESCRIPTION
<code>Runtime[ContextT]</code>	The runtime for the current graph run.